Computational Neuroscience — Week 7

Networks that Learn

Plasticity in the Brain and Learning

Last week the connection strengths W and M were handed to us. This week they become the things that change. Lecture 1 goes from two experimental phenomena — LTP and LTD — to a mathematical learning rule, asks whether that rule is stable, and ends by showing that a single neuron obeying it performs principal component analysis. Lecture 2 gives up on a single output neuron: competitive learning represents clusters instead of directions, self-organising maps add topography, and generative models with the EM algorithm put the whole business on a principled footing. Lecture 3 applies all of it to natural images, deriving sparse coding from a generative model and arriving at receptive fields that look like those in V1. Green boxes give the plain-words version of each idea before the mathematics; rust boxes flag the places where the obvious reading is the wrong one.

Contents

I Synaptic Plasticity and Learning **2**

1 Where We Are **2**

2 Two Experimental Facts: LTP and LTD **3**

3 Hebb's Rule **3**

4 The Average Effect of the Hebb Rule **5**

5 The Covariance Rule **5**

6 Are These Rules Stable? **6**

7 Oja's Rule **7**

8 What Does Hebbian Learning Actually Do? **8**

9 The Brain Can Do Statistics **9**

10 Where PCA Fails **10**

11 Where This Leaves Us **11**

II Competitive Learning and Unsupervised Learning **12**

12 Representing Clusters with Neurons **12**

13 Updating the Weights **13**

14 The Competitive Learning Algorithm **13**

15 Self-Organising Maps **15**

16 Two Models, and a Question **15**

17 Generative Models 16

18 Clustering as a Mixture of Gaussians 16

19 The EM Algorithm 17

20 EM in Action 18

21 Where This Leaves Us 20

III Sparse Coding and Predictive Coding 20

22 Eigenvectors Strike Again 20

23 A Linear Model with Better Basis Vectors 21

24 The Generative Model: Likelihood 22

25 The Generative Model: A Sparse Prior 22

26 The Bayesian Objective 24

27 Maximising F by Gradient Ascent 25

28 Step 1: The Recurrent Network 25

29 Step 2: Learning G 26

30 What the Network Learns 26

31 Predictive Coding 27

32 Where This Leaves Us 28

Part I

Synaptic Plasticity and Learning

Lecture 1 — LTP and LTD, Hebb's rule, the covariance and Oja rules, stability, and Hebbian learning as PCA.

1. Where We Are

[Lecture slide omitted from the public copy.] Cajal's drawing of the hippocampus. The first experimental results on synaptic plasticity came from this structure, which is why it opens the lecture.

The idea in plain words

Weeks 2–4 asked what neurons *encode*. Week 5 asked how a single cell *works*. Week 6 wired cells into networks and showed what those networks *compute* — but every weight matrix

was simply given to us.
That is the gap this week closes. A network whose weights are fixed by the modeller has not learnt anything; it has been programmed. **Synaptic plasticity** is the mechanism by which the brain sets its own weights from experience, and this lecture turns it into equations.

2. Two Experimental Facts: LTP and LTD

[Lecture slide omitted from the public copy.] Long-term potentiation. Neuron A fires a burst while B is also active; afterwards the same input from A produces a visibly larger EPSP.

Long-term potentiation (LTP)

An experimentally observed **increase** in the synaptic strength from neuron A to neuron B that lasts for hours or days.
Induction: make A fire a burst of spikes *while* B is also depolarised or firing.
Readout: for the same single input from A, the excitatory postsynaptic potential (EPSP) recorded in B becomes larger.

[Lecture slide omitted from the public copy.] Long-term depression. The same input from A, but now B stays silent — and the EPSP shrinks.

Long-term depression (LTD)

An experimentally observed **decrease** in synaptic strength, also lasting hours or days.
Induction: make A fire *while* B *does not* fire and does not depolarise.
Readout: the EPSP for the same input becomes smaller.

The idea in plain words

The two phenomena are the same experiment with one variable flipped. The input is identical in both cases — neuron A fires. What differs is whether the *postsynaptic* cell responded. Input with a response strengthens the synapse. Input without a response weakens it. That single sentence is the whole of what follows; everything else is finding an equation that says it.

Watch out — the EPSP is the measurement, not the thing that changed

What is recorded is the size of the postsynaptic potential. What is *inferred* is a change in the strength of the connection. The experimentalist never sees a weight; they see a voltage and reason back to a weight, exactly as the modeller writes down a w and reasons forward to a voltage.

3. Hebb's Rule

[Lecture slide omitted from the public copy.] Hebb's learning rule, and the mantra it is usually reduced to.

The idea in plain words

The historically striking part is the order of events. Donald Hebb, a Canadian psychologist, proposed this rule in 1949 — *before* LTP or LTD had been observed in the brain. It was a prediction from psychology that the physiology later confirmed.

Hebb's learning rule

If a neuron A repeatedly takes part in firing another neuron B, then the synapse from A to B should be strengthened.
Or, in the form everyone quotes: *neurons that fire together, wire together.*

Watch out — "take part in firing" is stronger than "fire at the same time"

Hebb's own wording asks that A *participate in causing* B to fire — a causal claim, with A before B. The mantra flattens that into simple co-activity.
The distinction is not pedantic: it is exactly the difference between the rules in this lecture, which see only simultaneous firing rates, and spike-timing-dependent plasticity, where a synapse strengthens if A fires *before* B and weakens if it fires after. The mathematics below deliberately takes the weaker reading, because rates carry no order.

Turning the mantra into mathematics

Take the linear feedforward neuron from last week — input vector $\mathbf{u}$, weight vector $\mathbf{w}$, and firing-rate dynamics fast enough that we can use the steady-state output:

$$v = \mathbf{w} \cdot \mathbf{u} = \mathbf{w}^\top \mathbf{u} = \mathbf{u}^\top \mathbf{w}$$

The basic Hebb rule *the **LTP model***

$$\tau_w \frac{d\mathbf{w}}{dt} = \mathbf{u}\, v$$

where τ_w is a time constant governing how fast the weights change.

The idea in plain words

The rule is a **product**, and that is the entire trick. A product is large only when *both* factors are large. Presynaptic activity u_i alone gives nothing if $v = 0$; postsynaptic activity v alone gives nothing to a synapse whose $u_i = 0$.
So the synapse strengthens exactly when input and output are active together, which is what Hebb asked for — and it does so *locally*: the update to w_i needs only u_i and v, both of which are physically available at that synapse. No global error signal, nothing the neuron could not know.

Watch out — τ_w is not the membrane time constant

There are now three timescales in play: $\tau_m = R_m C_m$ for the membrane (milliseconds), τ_r for the firing rate, and τ_w for the weights. Learning is assumed to be much *slower* than activity — the weights are effectively constant while the neuron responds to any one input. That separation is what lets us use the steady-state $v = \mathbf{w} \cdot \mathbf{u}$ inside the learning rule at all.

The discrete update

To run this on a computer, discretise. With $\Delta t/\tau_w$ absorbed into a single positive constant ϵ, the **learning rate**:

$$\mathbf{w}_{i+1} \;=\; \mathbf{w}_i + \epsilon\,\mathbf{u}v \qquad \text{or equivalently} \qquad \Delta\mathbf{w} \;=\; \epsilon\,\mathbf{u}v$$

The idea in plain words

This is the form you will recognise from machine learning. It is also, quite literally, the ancestor of it — the perceptron learning rule and everything downstream of it descend from this line.

4. The Average Effect of the Hebb Rule

A single input tells us little; what matters is what the rule does over the whole ensemble of inputs. Average the right-hand side over $\mathbf{u}$, writing $\langle\cdot\rangle$ for that average, and substitute $v = \mathbf{u}^\top\mathbf{w}$:

$$\tau_w \frac{d\mathbf{w}}{dt} \;=\; \langle \mathbf{u}v \rangle \;=\; \langle \mathbf{u}\,\mathbf{u}^\top\mathbf{w} \rangle \;=\; \langle \mathbf{u}\mathbf{u}^\top \rangle\,\mathbf{w} \;=\; Q\,\mathbf{w}$$

The input correlation matrix

$$Q \;=\; \langle \mathbf{u}\mathbf{u}^\top \rangle$$

Entry $Q_{ij} = \langle u_i u_j \rangle$ is the average product of inputs i and j — how strongly those two inputs tend to be active together.

Watch out — w comes out of the average, u does not

The step $\langle \mathbf{u}\mathbf{u}^\top\mathbf{w} \rangle = \langle \mathbf{u}\mathbf{u}^\top \rangle\mathbf{w}$ is only legitimate because $\mathbf{w}$ is treated as constant while we average over inputs. That is the slow-learning assumption again, now doing real work in a derivation rather than just sounding reasonable.

The idea in plain words

The result is worth pausing on. We wrote down a rule about *individual* events — this input fired, that output fired — and on average it turns out to be driven by a *statistic* of the input ensemble.

The neuron does not know Q. No structure in the brain computes Q and hands it over. Yet a synapse following a purely local rule ends up being moved as though multiplied by it. What $Q\mathbf{w}$ actually does to $\mathbf{w}$ is the question the lecture leaves hanging until the end.

5. The Covariance Rule

The basic Hebb rule can only ever *increase* weights: $u_i \geq 0$ and $v \geq 0$ are firing rates, so $u_i v \geq 0$ always. It models LTP and has no way to express LTD.

The covariance rule — *the **LTP and LTD model***

$$\tau_w \frac{d\mathbf{w}}{dt} \;=\; \mathbf{u}\,\big(v - \langle v \rangle\big)$$

The idea in plain words

The threshold is no longer zero but the neuron's own *average* output.

- $v > \langle v \rangle$: the bracket is positive, weights grow ⇒ **LTP**
- $v < \langle v \rangle$, including $v = 0$: the bracket is negative, weights shrink ⇒ **LTD**

One rule, both phenomena, and the switch between them is a comparison the neuron can plausibly make against its own recent activity.

Averaging as before:

$$\tau_w \frac{d\mathbf{w}}{dt} = \big(\langle \mathbf{u}\mathbf{u}^\top \rangle - \langle \mathbf{u} \rangle \langle \mathbf{u} \rangle^\top\big)\mathbf{w} = C\,\mathbf{w}$$

The input covariance matrix

$$C = \langle \mathbf{u}\mathbf{u}^\top \rangle - \langle \mathbf{u} \rangle \langle \mathbf{u} \rangle^\top$$

the correlation matrix with the effect of the mean removed. This is why the rule is called the covariance rule.

Watch out — $C = Q$ only when the input mean is zero

If $\langle \mathbf{u} \rangle = \mathbf{0}$ the two matrices coincide and the two rules do the same thing on average. Since firing rates are non-negative, real input means are *never* zero — so the distinction is not a corner case, it is the normal situation, and §8 shows exactly what it costs.

6. Are These Rules Stable?

A rule that drives $\|\mathbf{w}\|$ to infinity is useless: real synapses have finite strength. The test is to watch $\|\mathbf{w}\|^2$ over time.

The Hebb rule

$$\frac{d\|\mathbf{w}\|^2}{dt} = 2\,\mathbf{w} \cdot \frac{d\mathbf{w}}{dt} = \frac{2}{\tau_w}\,\mathbf{w}^\top \mathbf{u}\, v = \frac{2}{\tau_w} v^2 \geq 0$$

The idea in plain words

The middle step is the one to notice: $\mathbf{w}^\top \mathbf{u}$ is just v again, so the whole expression collapses to v^2 — and a square cannot be negative.

Unless the neuron is silent for all time, $\|\mathbf{w}\|$ increases at every moment. $\mathbf{w}$ grows without bound. The rule is **unstable**.

The covariance rule

One might hope that adding LTD fixes this — the rule can now decrease weights, so perhaps the increases and decreases balance. They do not. Taking the same derivative and averaging gives $d\|\mathbf{w}\|^2/dt > 0$ again.

Watch out — LTD is not the same as stability

This is the lecture's first genuine surprise. A rule that can push weights *down* still grows without bound, because the weights that get pushed down are not the same ones that get pushed up. Growth along the favoured direction outruns shrinkage everywhere else, and the length is dominated by the winner.
Being able to decrease individual weights says nothing about the behaviour of the vector as a whole.

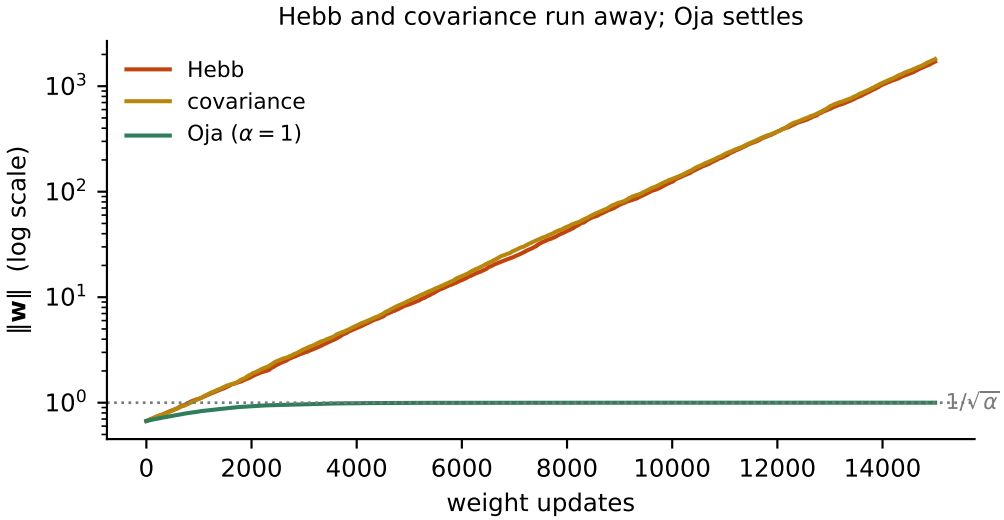


Figure 1: *Both unstable rules grow as straight lines on a log axis — that is exponential growth — reaching* $\|\mathbf{w}\| \sim 10^3$ *within 15,000 updates. Oja's rule settles at* $1/\sqrt{\alpha}$ *and stays there.*

Fixing it by hand

The blunt fix is to renormalise after every update, dividing $\mathbf{w}$ by its own length so that $\|\mathbf{w}\| = 1$ always.

The idea in plain words

It works, but it is a **hack**: an external operation imposed on the synapses from outside, requiring every synapse of the neuron to be rescaled simultaneously by a quantity none of them individually knows. Biologically that is hard to justify. We wanted a local rule and have quietly smuggled in a global one.

7. Oja's Rule

Oja's rule *the* ***stable model***

$$\tau_w \frac{d\mathbf{w}}{dt} = \mathbf{u}v - \alpha v^2 \mathbf{w}, \qquad \alpha > 0$$

The idea in plain words

The Hebb rule with a **decay term** bolted on. The subtracted piece is proportional to $\mathbf{w}$ itself, so it always pulls the weight vector back toward the origin, and it is scaled by v^2, so it pulls harder exactly when the Hebbian term is pushing hardest.
Crucially, both terms use only $\mathbf{u}$, v and $\mathbf{w}$ — all available at the synapse. Normalisation is no longer an external correction; it is built into the rule.

Taking the same derivative,

$$\frac{d\|\mathbf{w}\|^2}{dt} = \frac{2}{\tau_w} v^2\big(1 - \alpha\|\mathbf{w}\|^2\big)$$

Setting this to zero, and assuming $v \neq 0$, the bracket must vanish:

$$\|\mathbf{w}\|^2 = \frac{1}{\alpha} \qquad \Longrightarrow \qquad \|\mathbf{w}\| = \frac{1}{\sqrt{\alpha}}$$

The idea in plain words

And this fixed point is *stable*, not merely stationary. If $\|\mathbf{w}\|^2 < 1/\alpha$ the bracket is positive and the length grows; if it is larger the bracket is negative and the length shrinks. The rule corrects in both directions, so $\mathbf{w}$ converges to a sphere of radius $1/\sqrt{\alpha}$ from wherever it starts.
Note what α is doing: it does not affect *which direction* is learnt, only how long the resulting vector is. It sets the scale, not the answer.

Summary of the three rules

Rule	Equation	Models	Stability
Hebb	$\tau_w\dot{\mathbf{w}} = \mathbf{u}v$	LTP only	unstable*
Covariance	$\tau_w\dot{\mathbf{w}} = \mathbf{u}(v - \langle v\rangle)$	LTP and LTD	unstable*
Oja	$\tau_w\dot{\mathbf{w}} = \mathbf{u}v - \alpha v^2\mathbf{w}$	LTP and LTD	**stable**, $\|\mathbf{w}\| \to 1/\sqrt{\alpha}$

*unless a constraint on $\|\mathbf{w}\|$ is imposed by hand.

8. What Does Hebbian Learning Actually Do?

Now solve the averaged Hebb rule $\tau_w\, d\mathbf{w}/dt = Q\mathbf{w}$ properly.

The idea in plain words

The method is the one from last week: eigenvectors. $Q = \langle \mathbf{u}\mathbf{u}^\top \rangle$ is real and symmetric by construction, so its eigenvectors $\mathbf{e}_i$ are orthonormal and *any* vector can be written in that basis — including $\mathbf{w}(t)$.

Writing $\mathbf{w}(t) = \sum_i c_i(t)\,\mathbf{e}_i$ and substituting decouples the system into one independent equation per mode:

$$\tau_w \frac{dc_i}{dt} = \lambda_i c_i \qquad \Longrightarrow \qquad c_i(t) = c_i(0)\, e^{\lambda_i t/\tau_w}$$

$$\mathbf{w}(t) = \sum_i c_i(0)\, e^{\lambda_i t/\tau_w}\, \mathbf{e}_i$$

The idea in plain words

Every coefficient grows exponentially — the rule is unstable, we knew that. But they grow at *different* rates, set by their eigenvalues. As t becomes large the term with the largest eigenvalue λ_1 outgrows all the others by an exponentially widening margin, and the sum is eventually indistinguishable from that one term:

$$\mathbf{w}(t) \;\propto\; \mathbf{e}_1$$

Under Oja's rule the length is pinned as well, giving $\mathbf{w} \to \mathbf{e}_1/\sqrt{\alpha}$.

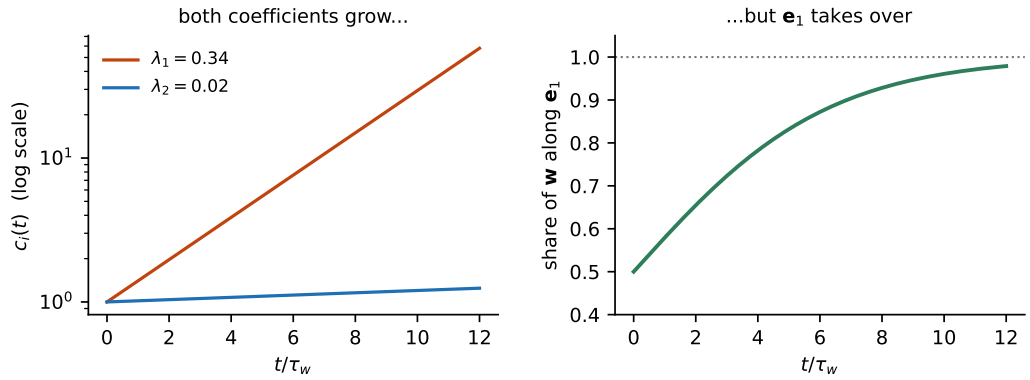


Figure 2: *Left: both coefficients grow without bound. Right: what matters is their* ratio *— the share of* $\mathbf{w}$ *lying along* $\mathbf{e}_1$ *goes to 1, so the direction converges even though the length does not.*

Watch out — direction and length converge on different schedules

This is why the instability is less damaging than it first appears. The *direction* of $\mathbf{w}$ settles down quickly and permanently; only its *magnitude* runs away. Oja's rule does not change the answer the network finds — it just stops the numbers from exploding while it finds it.

9. The Brain Can Do Statistics

The result

Hebbian learning drives $\mathbf{w}$ to the principal eigenvector of the input correlation matrix (Hebb rule) or covariance matrix (covariance rule) — that is, to the **direction of maximum variance** in the input data.
This is exactly **principal component analysis**.

[Lecture slide omitted from the public copy.] The lecture's version: Hebb on centred data, Hebb on shifted data, and the covariance rule on the same shifted data.

The idea in plain words

Read the middle panel carefully, because it is the point of the figure. With the data shifted to a mean of $(2, 2)$, the plain Hebb rule points $\mathbf{w}$ straight up the diagonal — along the *mean*, not along the spread.
It is not malfunctioning. It is faithfully finding the top eigenvector of $Q = \langle \mathbf{u}\mathbf{u}^\top \rangle$, and when

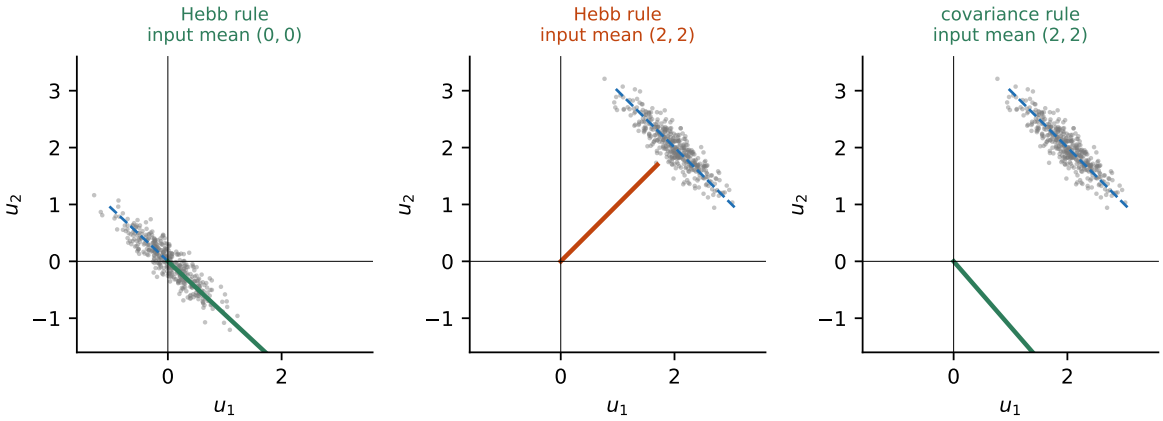


Figure 3: *The same three cases, simulated. Dashed blue is the true direction of maximum variance; the solid line is where the rule actually converged. Left and right land on it (within* $0.1°$ *and* $4°$*); the middle panel misses it by* $90°$*.*

the mean is large that matrix is dominated by the mean rather than by the variance. The covariance rule subtracts the mean first, and recovers the right answer.

Watch out — a rule can be correct and still useless

The Hebb rule answers the question it was asked — maximum *correlation*, measured from the origin. We wanted maximum *variance*, measured from the mean. Since firing rates are non-negative, the input mean is always positive and this failure is the default case, not an edge case.

This is also why the covariance rule earns its place. Its real justification is not that it can express LTD; it is that subtracting $\langle v \rangle$ is what makes the neuron sensitive to variation rather than to overall level.

Why this is worth having

The idea in plain words

PCA is the standard tool for **dimensionality reduction**. Project each data point onto the principal direction and a two-dimensional dataset becomes a single number, with the direction chosen to lose as little as possible.

Scale that up. The input might be the pixels of an image — tens of thousands of dimensions — and a handful of dominant eigenvectors may capture most of the structure in natural images or natural movies. A neuron doing this is performing a genuinely useful computation, learnt from its inputs alone, with no teacher and no error signal.

Together with last week's result that a network can compute derivatives, the lecture's summary is that the brain can do calculus *and* statistics.

10. Where PCA Fails

[Lecture slide omitted from the public copy.] The closing question of the lecture: data in two clusters, and the single direction the covariance rule finds through them.

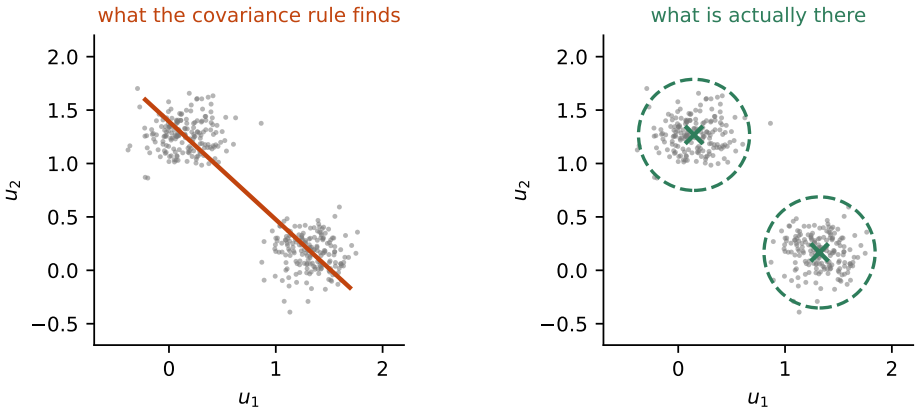


Figure 4: *Simulated. The learnt direction runs between the two clusters, describing neither, and passes through a region containing almost no data.*

The idea in plain words

The covariance rule does find the direction of maximum variance here — it is working correctly. But the variance it finds is the gap *between* the clusters, so the line it returns passes through empty space and describes no actual data point.
The failure is in the model, not the algorithm. PCA assumes the data form a single elongated blob, and asks only for one direction. Two clusters are a different kind of structure, and no single direction can express them: what you want is a statement about *membership*, not about *projection*.

Watch out — one output neuron can only learn one direction

Notice the constraint that has quietly been in force all lecture: there is a single output v, a single weight vector $\mathbf{w}$, and therefore a single learnable direction.
Two clusters need at least two units, and some mechanism that stops both from learning the same thing. That mechanism is competition, and it is where the next lecture goes.

11. Where This Leaves Us

- **LTP and LTD** are the same experiment with the postsynaptic response switched on or off. Input with a response strengthens; input without one weakens.
- **Hebb's rule** formalises that as a product $\mathbf{u}v$, which is local, biologically plausible, and the ancestor of the learning rules used throughout machine learning.
- **On average**, the Hebb rule follows the input correlation matrix Q and the covariance rule the input covariance matrix C — ensemble statistics that no part of the system ever explicitly computes.
- **Being able to weaken synapses does not confer stability.** Both rules explode; only Oja's rule, with its $-\alpha v^2\mathbf{w}$ decay term, converges, to $\|\mathbf{w}\| = 1/\sqrt{\alpha}$.
- **Direction converges even when length does not**, because the largest eigenvalue dominates the eigenvector expansion. This is why instability is survivable.
- **Hebbian learning is PCA**, finding the direction of maximum variance — but only the

covariance rule measures that from the mean rather than the origin, and with non-negative firing rates that distinction always matters.

- **One neuron learns one direction.** Clustered data defeat it, and fixing that requires competition between units.

> *To add from lectures: Next lecture: competitive learning, generative models, and unsupervised learning.*

Part II

Competitive Learning and Unsupervised Learning

Lecture 2 — representing clusters, competitive learning, self-organising maps, generative models, and the EM algorithm.

12. Representing Clusters with Neurons

The previous lecture ended with data in two clusters that no single weight vector could describe. The fix is to stop insisting on a single output neuron.

[Lecture slide omitted from the public copy.] Two output neurons, one per cluster, with each weight vector placed at a cluster centre — and the algebra showing that the closest weight vector belongs to the most active neuron.

The idea in plain words

The idea is disarmingly simple. Use *one output neuron per cluster*, and set that neuron's weight vector to the **centre** of its cluster.
The weight vector stops being an abstract direction and becomes a location — a **prototype**, a stored example of what that neuron's cluster looks like.

Why the closest weight vector wins

Expand the squared Euclidean distance between an input and a weight vector:

$$\|\mathbf{u} - \mathbf{w}_i\|^2 \;=\; \|\mathbf{u}\|^2 + \|\mathbf{w}_i\|^2 - 2\,\mathbf{w}_i^\top \mathbf{u} \;=\; \|\mathbf{u}\|^2 + \|\mathbf{w}_i\|^2 - 2v_i$$

The idea in plain words

If $\|\mathbf{u}\| = 1$ and $\|\mathbf{w}_i\| = 1$, the first two terms are constants shared by every neuron. Only $-2v_i$ varies, and it carries a minus sign, so

$$\arg\min_i \|\mathbf{u} - \mathbf{w}_i\|^2 \;=\; \arg\max_i v_i$$

The most active neuron *is* the one whose weights are closest to the input. This matters because the network cannot measure distances — it only has firing rates. The algebra says that firing rate is enough.

Watch out — the normalisation is doing real work

Drop it and the equivalence fails immediately. Take $\mathbf{u} = (0.6, 0.8)$ with $\mathbf{w}_1 = (0.8, 0.6)$ and $\mathbf{w}_2 = (0, 3)$:

	v_1, v_2	distances	winner
as given	0.96, 2.40	0.283, 2.280	neuron 2 fires most, neuron 1 is closest
normalised	0.96, 0.80	0.283, 0.632	neuron 1 on both counts

Neuron 2 wins on activity purely by having a *long* weight vector, not a well-placed one. Without normalisation, "fire the most" rewards loudness rather than similarity — which is exactly the runaway growth of the last lecture reappearing as a failure of representation.

13. Updating the Weights

Given a new input $\mathbf{u}_t$, assign it to the nearest cluster and move that cluster's centre to include it. The centre is a **running average**, and the update follows from the definition of a mean:

$$\mathbf{w}_A(t) \;=\; \frac{1}{t}\sum_{i=1}^{t}\mathbf{u}_i \;=\; \frac{t-1}{t}\,\mathbf{w}_A(t-1) + \frac{1}{t}\,\mathbf{u}_t$$

which rearranges into

$$\boxed{\Delta\mathbf{w}_A \;=\; \epsilon\,(\mathbf{u}_t - \mathbf{w}_A)}$$

The idea in plain words

The update is a **difference**, not a product — and that is what separates this lecture from the last one. Hebbian rules multiply activities together and grow without bound. This rule measures how far the input is from the current prototype and moves a fraction ϵ of the way there.

It cannot run away: once $\mathbf{w}$ sits at the centre of its cluster, the differences from surrounding points cancel on average and the weight stops moving. Stability comes free.

Watch out — $\epsilon = 1/t$ and ϵ constant are different algorithms

Setting $\epsilon = 1/t$ computes the exact running average — but $1/t \to 0$, so the network eventually stops learning and is deaf to anything new.

Holding ϵ at a small constant keeps it permanently **adaptive**, tracking a distribution that drifts, at the cost of never fully settling. Which you want depends on whether the world is stationary. For a brain, it is not.

14. The Competitive Learning Algorithm

Competitive learning — *the online algorithm*

For each input $\mathbf{u}$:

1. **Compete.** Pick the most active neuron — equivalently, the one whose weight vector is closest to $\mathbf{u}$. A winner-take-all recurrent network from last week can do this.
2. **Update the winner only.** $\Delta\mathbf{w} = \epsilon\,(\mathbf{u} - \mathbf{w})$.

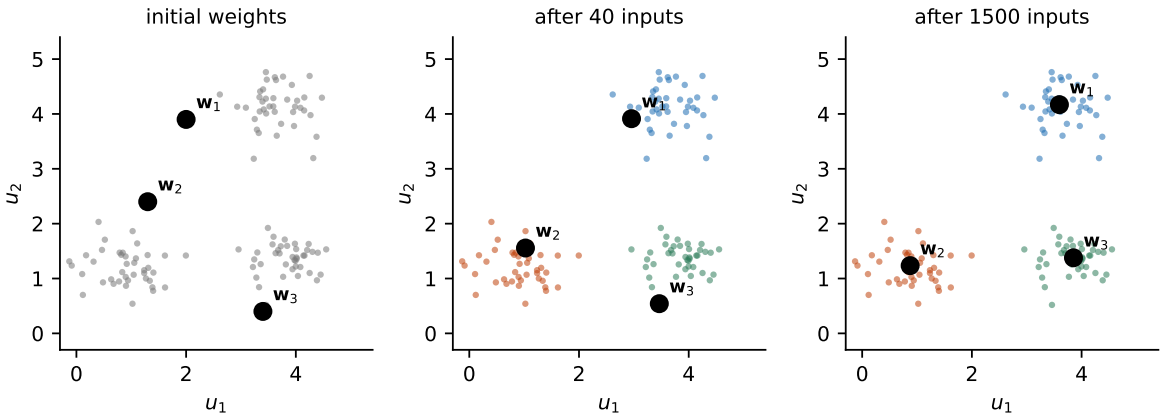


Figure 5: *Competitive learning run on three blobs. Weights start in the wrong places; after 1500 single-input updates each sits on a cluster centre, within* 0.09–0.13 *of the true one. Colours show which neuron owns which points.*

The idea in plain words

Notice how little the algorithm is told. It is never given the number of points per cluster, never shown a label, never handed an error signal. It sees one input at a time and adjusts one weight vector. The partition in the right-hand panel is something it discovered. This is a neural implementation of what statisticians call k-means.

Watch out — updating only the winner has a failure mode

A neuron that never wins never updates — and a neuron that never updates never moves closer to the data, so it never wins. The situation is self-perpetuating.

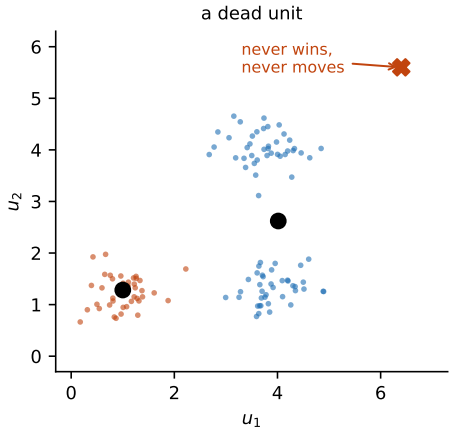


Figure 6: *A third unit initialised away from the data. After 1500 inputs it has moved a distance of exactly* 0 *and owns* 0 *points, and the remaining two units are left to cover three clusters.*

The idea in plain words

This is the **dead unit** problem, and it is a direct consequence of winner-take-all. The competition is decided by proximity, but only winners get to improve their proximity, so initialisation permanently determines who participates.

Every standard fix works by weakening the all-or-nothing rule — initialising weights on actual data points, penalising units that win too often, or letting losers update a little too. That last option is the one the rest of this lecture pursues.

15. Self-Organising Maps

[Lecture slide omitted from the public copy.] A Kohonen map morphing to cover a data cloud, and the orientation preference map in V1.

Self-organising map (Kohonen map)

Competitive learning with one addition: the neurons are given **locations** on a grid, and each input updates the winner *and its neighbours on that grid.*

The idea in plain words

The grid is a second space, separate from the input space. Each neuron now has two positions: where it sits on the grid, which is fixed, and where its weight vector sits in input space, which learns.
Because neighbours are dragged along with the winner, neurons that are close on the grid end up with similar weight vectors. The map becomes **topology-preserving**: a possibly very high-dimensional input space is laid out on a two-dimensional sheet in a way that keeps nearby things nearby.

The idea in plain words

And cortex appears to do exactly this. In V1, **orientation preference maps** vary smoothly across the cortical surface — move a short distance and the preferred orientation shifts a little, not randomly.
That is a real constraint the brain faces and the algorithm shares: cortex is a two-dimensional sheet with expensive long-range wiring, so a representation that keeps similar features physically close is cheap to wire. The same pressure produces the same solution.

Watch out — the map preserves neighbourhoods, not distances

A self-organising map guarantees that nearby grid locations code similar inputs. It does *not* guarantee that equal distances on the grid mean equal distances in input space — dense regions of the data get expanded and sparse regions compressed. The morphing grid in the slide is uneven for precisely this reason.

16. Two Models, and a Question

[Lecture slide omitted from the public copy.] PCA finds a direction; competitive learning finds clusters. Both are reasonable, and nothing so far says which to use.

The idea in plain words

We now have two ways of modelling data, arrived at by two different routes, with no principle connecting them. PCA came from a Hebbian rule and returns a direction. Competitive learning came from a running average and returns prototypes.
Which is right for a given dataset? So far, the honest answer is: whichever looked better in the pictures. That is not a theory. **Unsupervised learning** is the framework that turns the choice into a modelling decision.

17. Generative Models

[Lecture slide omitted from the public copy.] The generative picture: hidden causes v drawn from a prior, generating data $\mathbf{u}$ through a likelihood, both governed by parameters G.

The generative model

Assume the observed inputs $\mathbf{u}$ are produced by hidden **causes** v:

- a cause is drawn from the **prior** $p[v; G]$
- the data are drawn from the **likelihood** $p[\mathbf{u} \mid v; G]$

Unsupervised learning is the problem of learning the parameters G of both distributions from the data alone.

The idea in plain words

The shift here is worth stating plainly. Instead of asking "what feature should I extract from this data", we ask "what process could have *produced* this data, and what were its settings". The goal is to *mimic the data-generation process*. Once you have a model that could have generated the data, dimensionality reduction, clustering, denoising and generating new samples all fall out of it, rather than each needing its own algorithm.

Watch out — the arrows run in opposite directions

The generative model runs *causes* $\rightarrow$ *data*; that is the direction the world is assumed to work in. Perception has to run *data* $\rightarrow$ *causes*, which is the inverse problem, and the inverse is what Bayes' rule computes.
This is the encoding/decoding split from weeks 2 and 3, promoted from a pair of analysis techniques to the organising principle of learning. And it is circular in a productive way: recognising the causes requires knowing the parameters, while learning the parameters requires knowing the causes.

The two sub-problems

1. **Recognition.** Given $\mathbf{u}$, estimate the causes — that is, compute the posterior $p[v \mid \mathbf{u}; G]$.
2. **Learning.** Given those posteriors, update the parameters G.

18. Clustering as a Mixture of Gaussians

For the two-cluster data, the natural generative model is: pick one of two Gaussians according to a prior, then draw a point from it.

Mixture of Gaussians *the **generative model***

$$p[\mathbf{u}; G] \;=\; \sum_v p[\mathbf{u} \mid v; G]\; p[v; G] \;=\; \sum_v \mathcal{N}(\mathbf{u}; \boldsymbol{\mu}_v, \sigma_v I)\; \gamma_v$$

with parameters $G = (\boldsymbol{\mu}_v, \sigma_v, \gamma_v)$ — a mean, a width, and a prior probability for each cluster.

The idea in plain words

Compare this with what competitive learning learned. That algorithm produced the $\boldsymbol{\mu}_v$ and nothing else. The generative model additionally asks how *spread out* each cluster is (σ_v) and how *common* it is (γ_v).
Those extra parameters are not decoration. A wide cluster should claim points further away than a narrow one, and a common cluster should be the default explanation for an ambiguous point. Competitive learning, judging on distance alone, can express neither.

19. The EM Algorithm

[Lecture slide omitted from the public copy.] The two steps, iterated to convergence.

Expectation–maximisation *the **batch algorithm***

E step — compute the posterior over causes for every data point, by Bayes' rule:

$$p[v \mid \mathbf{u}; G] \;=\; \frac{\mathcal{N}(\mathbf{u}; \boldsymbol{\mu}_v, \sigma_v I)\, \gamma_v}{\sum_{v'} \mathcal{N}(\mathbf{u}; \boldsymbol{\mu}_{v'}, \sigma_{v'} I)\, \gamma_{v'}}$$

M step — re-fit the parameters, weighting every data point by that posterior:

$$\boldsymbol{\mu}_v = \frac{\sum_{\mathbf{u}} p[v \mid \mathbf{u}]\, \mathbf{u}}{\sum_{\mathbf{u}} p[v \mid \mathbf{u}]}, \qquad \sigma_v^2 = \frac{\sum_{\mathbf{u}} p[v \mid \mathbf{u}]\, \|\mathbf{u} - \boldsymbol{\mu}_v\|^2}{\sum_{\mathbf{u}} p[v \mid \mathbf{u}]}, \qquad \gamma_v = \frac{\sum_{\mathbf{u}} p[v \mid \mathbf{u}]}{N_u}$$

The idea in plain words

Each M-step formula is an ordinary weighted average, and the weights are the posteriors from the E step. The mean of cluster v is the average of all the data, weighted by how much each point belongs to v; its variance is the weighted average squared distance; its prior is simply the total responsibility it claimed, divided by the number of points.
Every point contributes to every cluster — just by different amounts.

Watch out — the variance formula needs a factor the slide omits

For an isotropic Gaussian in d dimensions, $\|\mathbf{u} - \boldsymbol{\mu}\|^2$ sums the squared deviation over all d axes, while σ^2 is the variance *per axis*. The maximum-likelihood estimate therefore carries an extra $1/d$:

$$\sigma_v^2 = \frac{1}{d} \frac{\sum_{\mathbf{u}} p[v \mid \mathbf{u}]\, \|\mathbf{u} - \boldsymbol{\mu}_v\|^2}{\sum_{\mathbf{u}} p[v \mid \mathbf{u}]}$$

In two dimensions the slide's version overestimates σ by a factor of $\sqrt{2}$. With the $1/d$ included, the simulation below recovers $\sigma = 0.181$ against a true isotropic equivalent of 0.181; without it the fitted circles are visibly too wide. The omission does not affect the

means or the qualitative story, which is presumably why it survived onto the slide.

Soft competition

The idea in plain words

The E step is competition, but of a different kind. Competitive learning declares a single winner that takes everything. The E step distributes **responsibility** across all the causes in proportion to how well each explains the point.
A point in the middle of cluster A gets a posterior near 1 for A; a point between the clusters gets something near 0.5 and genuinely belongs to both. Winner-take-all cannot represent that, and forcing a choice throws away exactly the information about which assignments were uncertain.

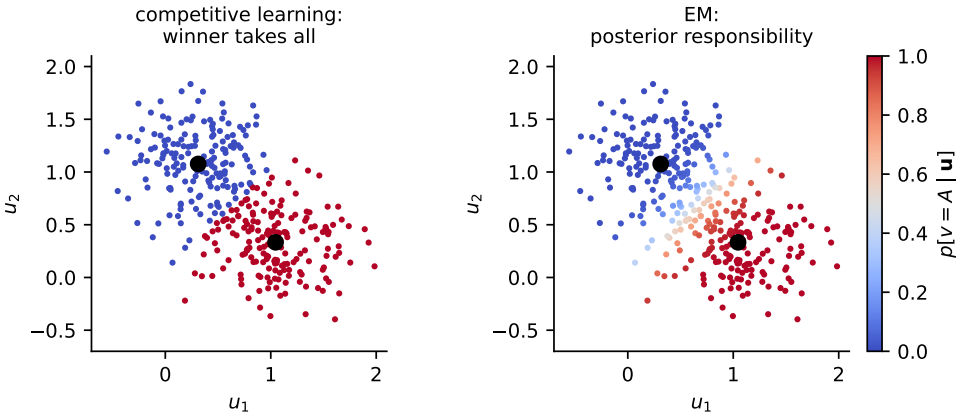


Figure 7: *The same overlapping data under both schemes. Winner-take-all splits them along a hard line; EM leaves a band of pale, genuinely ambiguous points — 63 of 360 have a posterior between 0.1 and 0.9.*

20. EM in Action

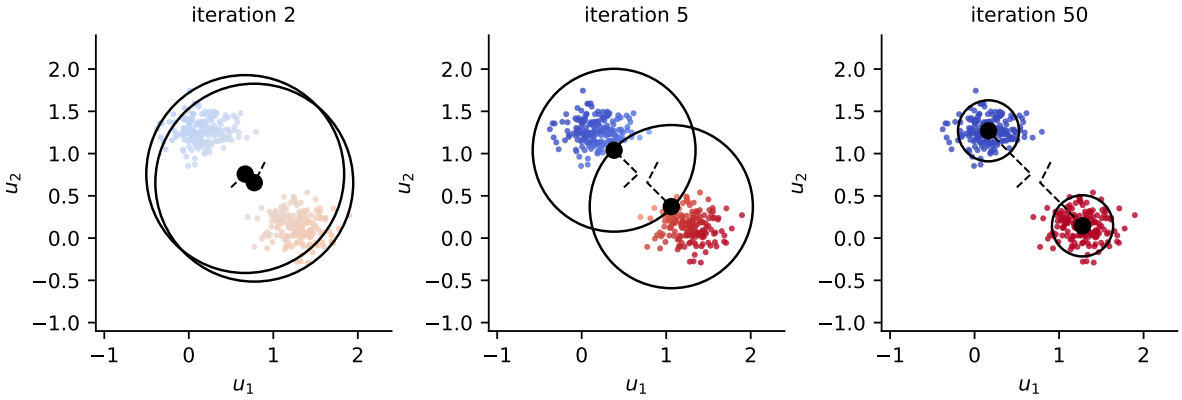


Figure 8: *EM on the two-cluster data, from a deliberately poor start. Dashed lines trace each mean; circles are two standard deviations. By iteration 50 the means and widths have both converged.*

The idea in plain words

Watch the widths as much as the means. At iteration 2 the model is one blob pretending to be two, with both circles enormous because a single wide Gaussian is the best available excuse for data it cannot yet separate.
As the means separate, each Gaussian needs to explain less, so its σ shrinks; and as σ shrinks the posteriors sharpen, which pulls the means apart faster still. Recognition and learning bootstrap each other — which is what makes the circularity noted earlier productive rather than fatal.

Competitive learning versus EM

	Competitive learning	**EM**
Assignment	winner-take-all (hard)	posterior responsibility (soft)
Learns	means only	means, variances *and* priors
Data	one input at a time (**online**)	all data at once (**batch**)
Basis	a plausible heuristic	maximum likelihood

Watch out — online versus batch is a real trade-off, not a ranking

EM is the better statistical procedure, but it needs the whole dataset in hand for every iteration. Competitive learning updates from a single input and never stores anything.
For a brain, which receives inputs one at a time and cannot replay its history, the online algorithm is the biologically feasible one. The batch algorithm is the one that tells you what the online algorithm is approximating.

The idea in plain words

For a brain the trade is obviously worth making. The world is not stationary, and a neuron that stopped learning after childhood would be useless. This is the same choice as $1/t$ versus constant step sizes in stochastic gradient descent, where constant rates are standard for exactly this reason.

Watch out — this is Bayes' rule run backwards

You normally use Bayes to get the *posterior* from the likelihood — that is the E step. Here it is rearranged to give the *likelihood* from the posterior.
It is the same identity either way, but the direction is worth noticing, because the generative model runs causes $\rightarrow$ data while recognition runs data $\rightarrow$ causes, and Bayes' rule is the single hinge between them.

The idea in plain words

This innocuous question is the setup for the next lecture. Every dataset in this lecture has been two-dimensional and drawable on a page. A single natural image is one point in a 10,000-dimensional space.
That is why "what generative model suits natural images" is a real question rather than a rhetorical one: mixtures of Gaussians and a handful of cluster prototypes are hopeless at that dimensionality, and the answer will have to look quite different.

21. Where This Leaves Us

- **One neuron per cluster, weight vector at the centre.** The weight vector becomes a prototype rather than a direction.
- **The most active neuron is the closest one** — but only if inputs and weights are normalised, otherwise activity rewards magnitude instead of similarity.
- **The update is a difference, not a product**, which is why competitive learning is stable where Hebbian rules were not.
- **Only the winner learns**, and that produces dead units: a badly-placed neuron can never win, so it can never improve.
- **Self-organising maps** update the winner's neighbours too, producing topology-preserving maps of the kind found in V1.
- **Generative models** reframe the problem: not what feature to extract, but what process produced the data. Recognition is then the inverse problem, solved by Bayes' rule.
- **EM alternates recognition and learning** — soft posteriors in the E step, weighted re-fitting in the M step — and learns spreads and priors that competitive learning cannot express.

> *To add from lectures: Next lecture: what generative model suits natural images, and how to learn the causes of such data.*

Part III

Sparse Coding and Predictive Coding

Lecture 3 — eigenfaces and their limits, sparse priors, the sparse coding network, Olshausen and Field's result, and predictive coding in the visual cortex.

22. Eigenvectors Strike Again

The question left hanging was how to model natural images. The obvious first move is the one that has worked all term: try eigenvectors.

[Lecture slide omitted from the public copy.] Turk and Pentland's eigenfaces — the principal eigenvectors of the covariance matrix of a set of face images.

The idea in plain words

Take many face images, each of N pixels, and compute the eigenvectors of the input covariance matrix. Because that matrix is real and symmetric, its eigenvectors form an orthonormal basis, so *any* face can be written as a linear combination of them:

$$\mathbf{u} = \sum_{i=1}^{N} \mathbf{e}_i v_i$$

These eigenvectors are themselves image-shaped, and they look like ghostly faces — hence **eigenfaces**.

Keep only the M eigenvectors with the largest eigenvalues and the equality becomes an approximation:

$$\mathbf{u} \;=\; \sum_{i=1}^{M} \mathbf{e}_i v_i + \text{noise}, \qquad M \ll N$$

The idea in plain words

The payoff is **compression**. For 1000×1000 images, $N = 10^6$. If the first 10 eigenvectors capture most of the variance then $M = 10$, and a million pixels are represented by ten numbers.

Watch out — good for compression, useless for parts

Eigenvectors let the course down here for the first time. PCA gives a compact *global* description, but it cannot extract the **local components** of an image — the eyes, the nose and the mouth of a face, or the edges in a natural scene.
The reason is structural. The eigenvectors are forced to be orthogonal and ordered by variance, and neither constraint has anything to do with being localised. The first component must explain as much of the whole image as possible, which pushes it towards being spread out over every pixel.

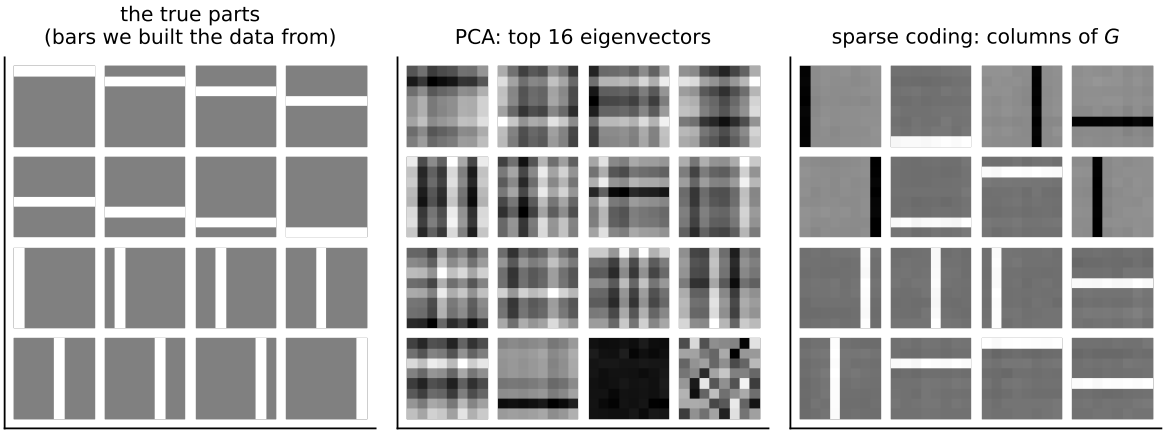


Figure 9: *Patches built from localised bars, then analysed two ways. PCA returns global, wave-like patterns; sparse coding recovers single bars. Measuring the fraction of each basis vector's energy falling in its best single row or column: true parts* 1.000, *PCA* 0.357, *sparse coding* 0.915. *Signs are arbitrary in sparse coding, so some bars come out dark.*

23. A Linear Model with Better Basis Vectors

Keep the linear model; drop the demand that the basis be eigenvectors.

$$\mathbf{u} \;=\; \sum_{i=1}^{M} \mathbf{g}_i v_i + \text{noise} \;=\; G\mathbf{v} + \text{noise}$$

where the columns of G are the basis vectors, or **features**, and $\mathbf{v}$ holds their coefficients.

The idea in plain words

One constraint is deliberately lifted: M may now be *larger* than N, the number of pixels. A basis with more vectors than dimensions is called **overcomplete**, and for an orthonormal basis it would be impossible.
It makes sense here because the number of possible parts of objects and scenes is much larger than the number of pixels in any one image. The dictionary should be big; what should be small is how much of it any single image uses.

Watch out — overcompleteness makes v ambiguous, on purpose

With $M > N$ the equation $\mathbf{u} = G\mathbf{v}$ has infinitely many solutions — there is no longer a unique way to write an image in terms of the features. That sounds like a defect.
It is what forces the rest of the lecture. Something extra has to pick among the solutions, and that something is the prior. Sparseness is not an afterthought bolted onto the model; it is what makes the model well-posed.

24. The Generative Model: Likelihood

Assume the noise is zero-mean Gaussian white noise — no correlations between components. Then the likelihood is a Gaussian centred on the reconstruction:

$$p[\mathbf{u} \mid \mathbf{v}; G] \;=\; \mathcal{N}(\mathbf{u};\, G\mathbf{v},\, I) \;\propto\; \exp\!\left(-\tfrac{1}{2}\|\mathbf{u} - G\mathbf{v}\|^2\right)$$

and taking logs leaves a single quadratic term:

$$\log p[\mathbf{u} \mid \mathbf{v}; G] \;=\; -\tfrac{1}{2}\|\mathbf{u} - G\mathbf{v}\|^2 + C$$

The idea in plain words

$\|\mathbf{u} - G\mathbf{v}\|^2$ is the squared **reconstruction error** — the difference between the actual image and the model's prediction of it.
So minimising squared reconstruction error, which is what a great many engineering and machine-learning algorithms do by default, *is* maximising the likelihood under Gaussian noise. The two are the same operation described in different vocabularies.

Watch out — the squared error is a modelling assumption, not a neutral choice

Squared error follows from assuming the noise is Gaussian. Assume a different noise distribution and a different error measure drops out — Laplacian noise gives absolute error, Poisson noise gives something else again.
Writing the model generatively makes that assumption visible. Reaching straight for least squares hides it.

25. The Generative Model: A Sparse Prior

Assume the causes are independent, so the prior factorises:

$$p[\mathbf{v}] = \prod_i p[v_i] \qquad \Longrightarrow \qquad \log p[\mathbf{v}] = \sum_i \log p[v_i]$$

The idea in plain words

This is a simplification and the lecture says so — components of natural images certainly do depend on one another. It is assumed anyway to see where it leads, which is a normal and honest way to build a model.

Now, what should $p[v_i]$ look like? If the causes are parts of scenes, then for any given image only a few parts are present and the rest of the v_i are zero.

A sparse (super-Gaussian) prior

A distribution with a **peak at zero** and a **heavy tail**: v_i is zero most of the time, but occasionally takes a large value.
Writing $p[v_i] \propto \exp(g(v_i))$, the lecture's two examples are

$$g(v) = -|v| \quad \text{(exponential)}, \qquad g(v) = -\log(1+v^2) \quad \text{(Cauchy)}$$

[Lecture slide omitted from the public copy.] The three distributions, and their logs.

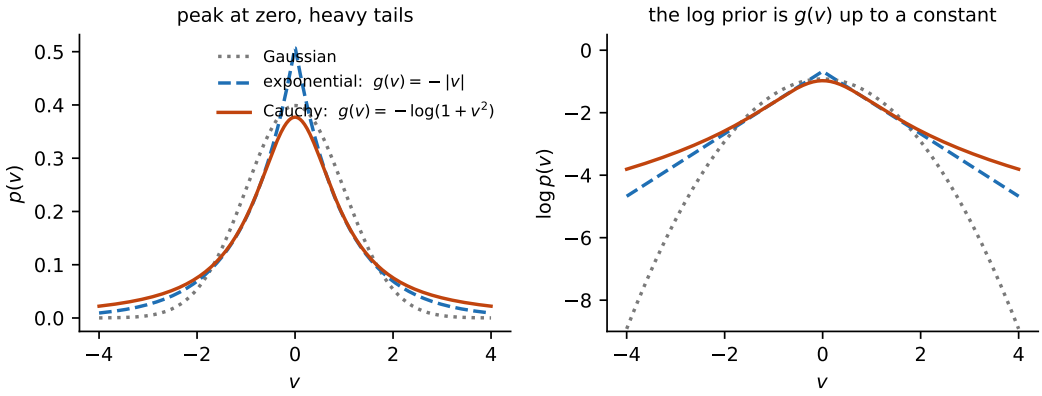


Figure 10: *The same three, computed. On the log axis the Gaussian is a parabola that plunges, while the sparse priors fall away far more gently — which is exactly what a heavy tail is.*

The idea in plain words

Put all three on a common robust scale and the difference is almost entirely in the tails, not at the peak. The mass sitting within half a median deviation of zero is nearly the same for all three (0.26–0.30). But the mass beyond five median deviations is 0.0008 for the Gaussian, 0.031 for the exponential and 0.125 for the Cauchy — a factor of over 150.
A Gaussian says large values essentially never happen. A sparse prior says they are rare but entirely possible. That is the whole difference, and it is what lets a few coefficients be large while the rest sit at zero.

Watch out — do not summarise these with kurtosis

Kurtosis is the usual number quoted for "super-Gaussian", but it is a poor choice here: the Cauchy distribution has no finite variance, so its kurtosis does not exist, and computing it on a truncated grid quietly discards the very tails it is meant to measure. My first attempt did exactly that and reported a Cauchy *less* heavy-tailed than an exponential, which is

backwards. The tail-mass figures above are used instead.

26. The Bayesian Objective

Maximise the posterior over causes. By Bayes, $p[\mathbf{v} \mid \mathbf{u}; G] = k\, p[\mathbf{u} \mid \mathbf{v}; G]\, p[\mathbf{v}; G]$, and taking logs turns the product into a sum:

The objective function *the log posterior*

$$F(\mathbf{v}, G) \;=\; \underbrace{-\tfrac{1}{2}\|\mathbf{u} - G\mathbf{v}\|^2}_{\text{reconstruction error}} \;+\; \underbrace{\sum_i g(v_i)}_{\text{sparseness}} \;+\; K$$

The idea in plain words

Two terms pulling in opposite directions. The first wants the reconstruction to be accurate, which is easiest when many coefficients are free to take whatever values help. The second wants most coefficients at zero, which makes accurate reconstruction harder.
Maximising F is negotiating between them: represent the image *well*, using *few* features.

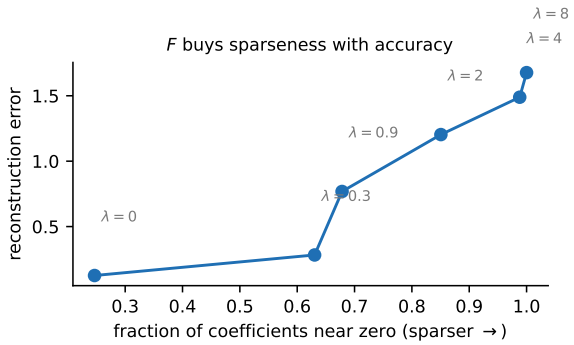


Figure 11: *Sweeping the weight on the sparseness term. Reconstruction error rises monotonically as the code is forced to be sparser — from error* 0.13 *with a quarter of coefficients near zero, to* 1.68 *with all of them. There is no setting that is best at both.*

Watch out — neither extreme is a model

With no sparseness term the network reconstructs beautifully and learns nothing interpretable — the objective collapses back towards ordinary least squares, and the recovered features are global, as the PCA panel showed. With the sparseness term dominant, the code is perfectly sparse because it is entirely zero, and represents nothing.
The useful behaviour is in between, and the weighting is a genuine modelling choice rather than something the data determine.

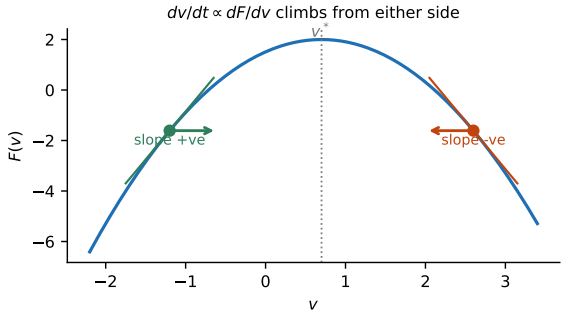


Figure 12: *Why following the gradient works from either side: left of the peak the slope is positive and v increases; right of it the slope is negative and v decreases.*

27. Maximising F by Gradient Ascent

The idea in plain words

Gradient ascent updates v in proportion to dF/dv. Left of the optimum the gradient is positive, so v grows; right of it the gradient is negative, so v shrinks. Either way the step is towards v^*, without anything needing to know where v^* is.

The two alternating steps

1. Maximise F with respect to $\mathbf{v}$, holding G fixed.
2. Maximise F with respect to G, using the $\mathbf{v}$ just found.

Structurally this is the EM algorithm again: step 1 is recognition (the E step), step 2 is learning (the M step).

28. Step 1: The Recurrent Network

Differentiating F with respect to $\mathbf{v}$:

$$\frac{d\mathbf{v}}{dt} \propto \frac{\partial F}{\partial \mathbf{v}} = G^\top \underbrace{(\mathbf{u} - G\mathbf{v})}_{\text{error}} + g'(\mathbf{v})$$

[Lecture slide omitted from the public copy.] The network implementing that equation: predict, subtract, correct.

The idea in plain words

Read the equation as a circuit and it describes something the network can actually do:

- $G\mathbf{v}$ — the network's **prediction** of its own input
- $\mathbf{u} - G\mathbf{v}$ — the **prediction error**
- $G^\top(\cdot)$ — the error sent back to the output layer
- $g'(\mathbf{v})$ — a pull towards zero, from the sparseness prior

For each image the network iterates: predict the input, measure how wrong the prediction is, correct the estimate of the causes, predict again. It settles on a stable $\mathbf{v}$.

This is firing-rate dynamics of exactly the kind from week 6 — the same $\tau\, d\mathbf{v}/dt = \ldots$ form. What is new is the *interpretation*: the recurrent dynamics are performing inference.

29. Step 2: Learning G

Differentiating F with respect to G gives the learning rule

$$\tau_G \frac{dG}{dt} = (\mathbf{u} - G\mathbf{v})\,\mathbf{v}^\top$$

Watch out — τ_G must be much larger than the time constant for v

The two steps only alternate properly if $\mathbf{v}$ settles before G moves. This is the same separation of timescales as in lecture 1, where learning had to be slow compared with activity — here it is what keeps the alternation from becoming a single tangled process.

The idea in plain words

Expand the rule and the first term is $\mathbf{u}\mathbf{v}^\top$ — plain **Hebbian** learning, presynaptic activity times postsynaptic activity. The second, $-G\mathbf{v}\mathbf{v}^\top$, is a subtractive term proportional to the weights themselves.
That is Oja's rule, almost exactly. A rule derived from Bayesian inference over a generative model turns out to be the biologically plausible local rule from the first lecture, with a decay term that arrives for free rather than being bolted on for stability.

Watch out — so why doesn't this just compute the eigenvectors?

It is a fair question, since Oja's rule provably finds the principal eigenvector. The answer is that the learning rule is not the whole algorithm: the $\mathbf{v}$ it is fed comes from step 1, which included the sparseness term $g'(\mathbf{v})$.
Same Hebbian weight update, different inputs to it, completely different outcome — localised features instead of global eigenvectors. The simulation above measures the difference: 0.915 against 0.357 on localisation.

30. What the Network Learns

[Lecture slide omitted from the public copy.] Olshausen and Field, 1996. Each square is one column of G, learnt from patches of natural images.

The idea in plain words

The learnt features are localised, oriented edge and bar detectors — and they look strikingly like the **receptive fields of simple cells in V1** that Hubel and Wiesel found by recording from cat cortex.
Nothing about V1 went into the model. The inputs were patches of natural images; the assumptions were a linear generative model, Gaussian noise and a sparse prior. Receptive fields resembling those of real neurons came out.

An interpretive model

This is the third kind of model from week 1, and the clearest example of it in the course. It does not describe *what* V1 encodes, nor the mechanism of *how* a cell responds. It offers a reason *why* the receptive fields have the form they do: they are what you get if the visual system is representing natural images sparsely and efficiently.

Watch out — resemblance is evidence, not proof

The result says sparse coding of natural images *suffices* to produce V1-like receptive fields. It does not establish that this is what the brain is doing — other objectives also yield oriented filters, and the model is silent about development, plasticity and the many V1 properties it does not address.
It is a strong existence proof and an elegant one. It is not a demonstration that cortex optimises this particular F.

31. Predictive Coding

[Lecture slide omitted from the public copy.] Sparse coding as a special case of a predictive coding network.

Predictive coding

- **Feedback** connections carry *predictions* of the activity in the area below.
- **Feedforward** connections carry the *error* between the prediction and what actually arrived.
- A "predictive estimator" maintains the estimate $\mathbf{v}$ of the hidden causes.

Optional additions: **recurrent** weights, letting the causes evolve over time so the model handles natural movies rather than static images; and a **gain** on the sensory error, which is one way of modelling visual attention.

The idea in plain words

The inversion here is worth stating plainly. In the standard picture, feedforward connections carry the stimulus upward and feedback modulates. In predictive coding, feedforward connections carry only what the higher area *failed to predict*.
A perfectly predicted input produces no feedforward signal at all. What travels up the hierarchy is surprise.

An anatomical puzzle

[Lecture slide omitted from the public copy.] Gilbert and Lee's diagram of visual cortical connectivity, and the schematic version of the puzzle.

The idea in plain words

The anatomy has long presented a puzzle: wherever there is a feedforward connection between two cortical areas — V1 to V2, say — there is almost always a reciprocal feedback connection from V2 back to V1. This holds throughout the visual hierarchy, retina to LGN to V1 to V2 and onward.
Feedback is not a rare embellishment; it is as ubiquitous as feedforward connectivity, and a functional account of cortex has to say what it is for.

The idea in plain words

Predictive coding gives each direction a specific job. Feedback carries predictions downward; feedforward carries errors upward. Reciprocity is then not a curiosity but a structural

requirement — the algorithm cannot run without both.
Trained on natural images, hierarchical predictive coding models also reproduce **contextual effects** such as surround suppression, where a neuron's response to a stimulus in its receptive field is modulated by what surrounds it. Under this account the surround is the context the higher area uses to predict the centre, so a predictable centre yields a reduced response.

32. Where This Leaves Us

- **PCA compresses but does not decompose.** Eigenfaces reduce a million pixels to ten numbers, yet orthogonality and variance-ordering prevent the components from being local parts.
- **Dropping orthogonality allows an overcomplete dictionary**, with more features than pixels — which makes $\mathbf{v}$ ambiguous, and that ambiguity is what the prior exists to resolve.
- **Gaussian noise makes squared reconstruction error the same thing as log likelihood**, so a familiar engineering objective turns out to be a statement about noise.
- **A sparse prior differs from a Gaussian almost entirely in the tails**, which is what permits a few large coefficients among many zeros.
- F **trades reconstruction against sparseness**, and neither extreme is a usable model.
- **The inference step is a recurrent network** that predicts its own input, measures the error, and corrects — week 6's dynamics doing Bayesian inference.
- **The learning step is Hebbian, essentially Oja's rule.** It escapes PCA only because the $\mathbf{v}$ it learns from was computed under a sparse prior.
- **The learnt features look like V1 simple cells**, an interpretive model explaining receptive field structure as efficient coding of natural images.
- **Predictive coding generalises this**, and assigns roles to the reciprocal feedforward and feedback connections that pervade visual cortex: errors up, predictions down.

To add from lectures: Next week: neurons as classifiers, and learning from rewards — reinforcement learning.